软件工程学习路线 | 26 年最新零基础到精通一条龙（万人收藏⭐）

> *编程导航学习网站：学编程、做项目、拿 Offer！*
>
> *企业高频面试题库：开始刷题，面试遇原题！*
>
> *精选简历模板大全：1 分钟搞定简历！*
>
> *AI 资源导航网站：获取最新 AI 黑科技！*
>
> *1 对 1 模拟面试：随时随地提升面试能力*

开篇介绍

软件工程是什么？

听起来好像挺唬人的，简单来说，它是一种系统性、规范化的软件开发方法论，就像建筑工程中的设计、施工、验收等阶段一样。开发一个软件就好比建造一座大楼，必须严格执行规划、设计、施工、检测等步骤，而且每个步骤都要遵循一定的规范、流程和方法，才能最终建造出符合要求的高楼大厦。否则可能你建楼建到一半，发现无法再盖更多层了。

软件工程强调在软件开发过程中，按照一定的流程和方法来组织、规划、管理和控制软件开发过程，以保证开发出高质量、可靠、易维护的软件项目。随着软件系统越来越复杂，团队协作越来越重要，软件工程的价值也越来越凸显。

为什么要学软件工程？

很多同学刚开始学编程时，都是自己一个人写代码，想怎么写就怎么写。但当你进入企业、参加团队开发时，就会发现：如果大家都不遵循规范和流程，代码会变得越来越混乱，随便加个新功能、老功能就没法用了，也就是所谓的“屎山代码”。学习软件工程，能让你理解如何在团队中高效协作，如何开发出高质量的软件，如何避免走弯路。

软件工程能解决什么问题？核心是 4 个字 —— 提质增效。提质，指的是通过规范的流程、方法和工具来提高软件质量，降低软件维护成本；增效，指的是通过设计模式、项目框架、自动化工具来提高开发效率，缩短软件开发周期。此外，软件工程还能提高团队协作效率、提高软件安全性、培养系统性思维。

需要注意的是，软件工程包含大量的理论知识，学起来可能会比较枯燥。鱼皮建议当你 **已经能够独立开发一个完整的项目后**，再抽时间系统学习软件工程。如果你是想快速找工作的同学，可以先忽略，因为面试基本不会直接考软件工程理论，但会考设计模式和开发规范。

在 AI 时代，软件工程的重要性不降反增。AI 应用开发同样需要遵循软件工程的规范和流程，从需求分析、系统设计到测试部署，每个环节都不能少。而且 AI 工具（如 Cursor、GitHub Copilot）能帮我们自动生成代码、自动测试、自动部署，让软件工程的实践变得更加高效。

举个例子，如果你让 AI 从 0 开发一个项目，最好也是先提供和确认方案，而不是无脑写代码：

编程导航(codefather.cn)



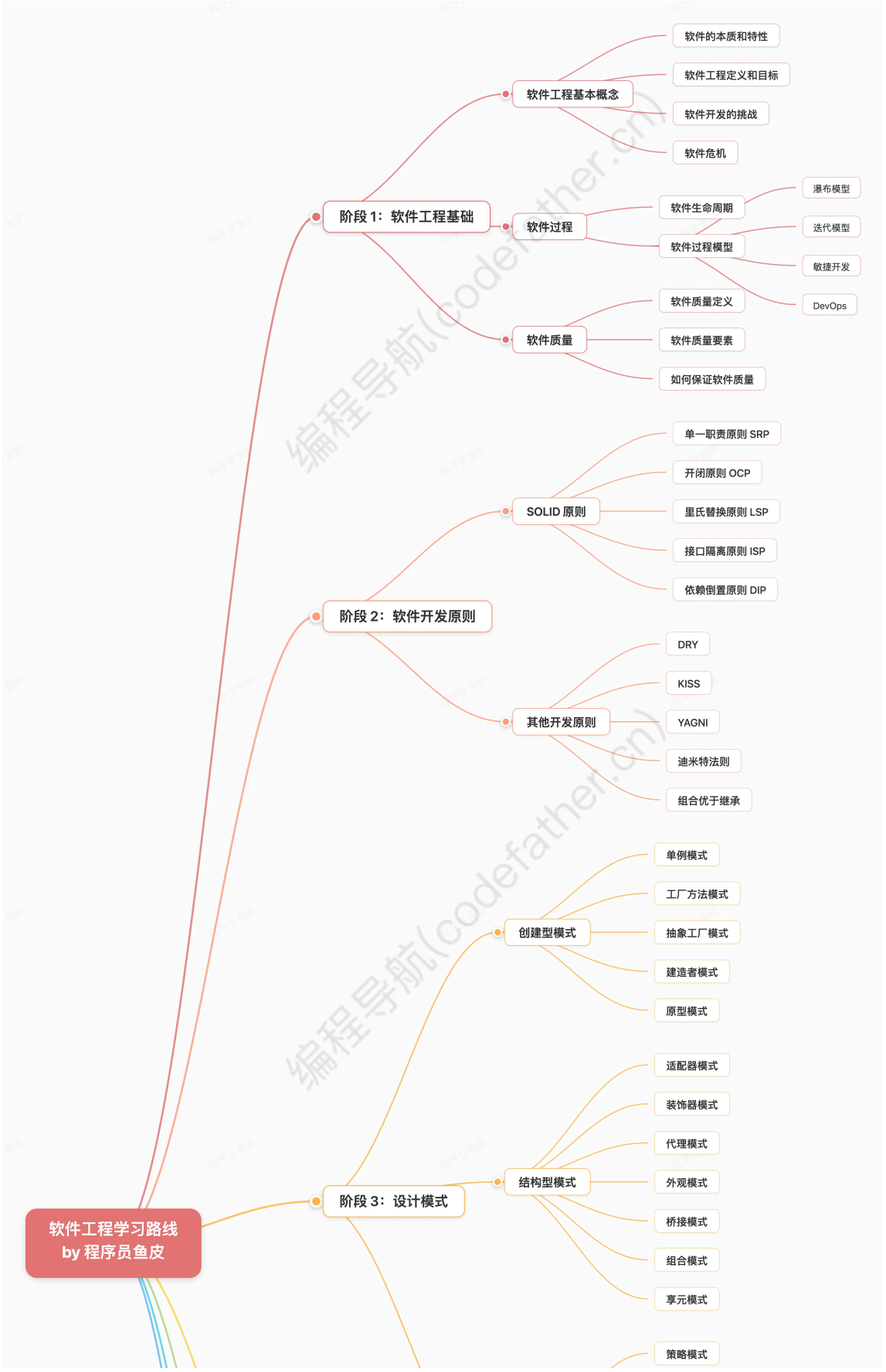
就业方向

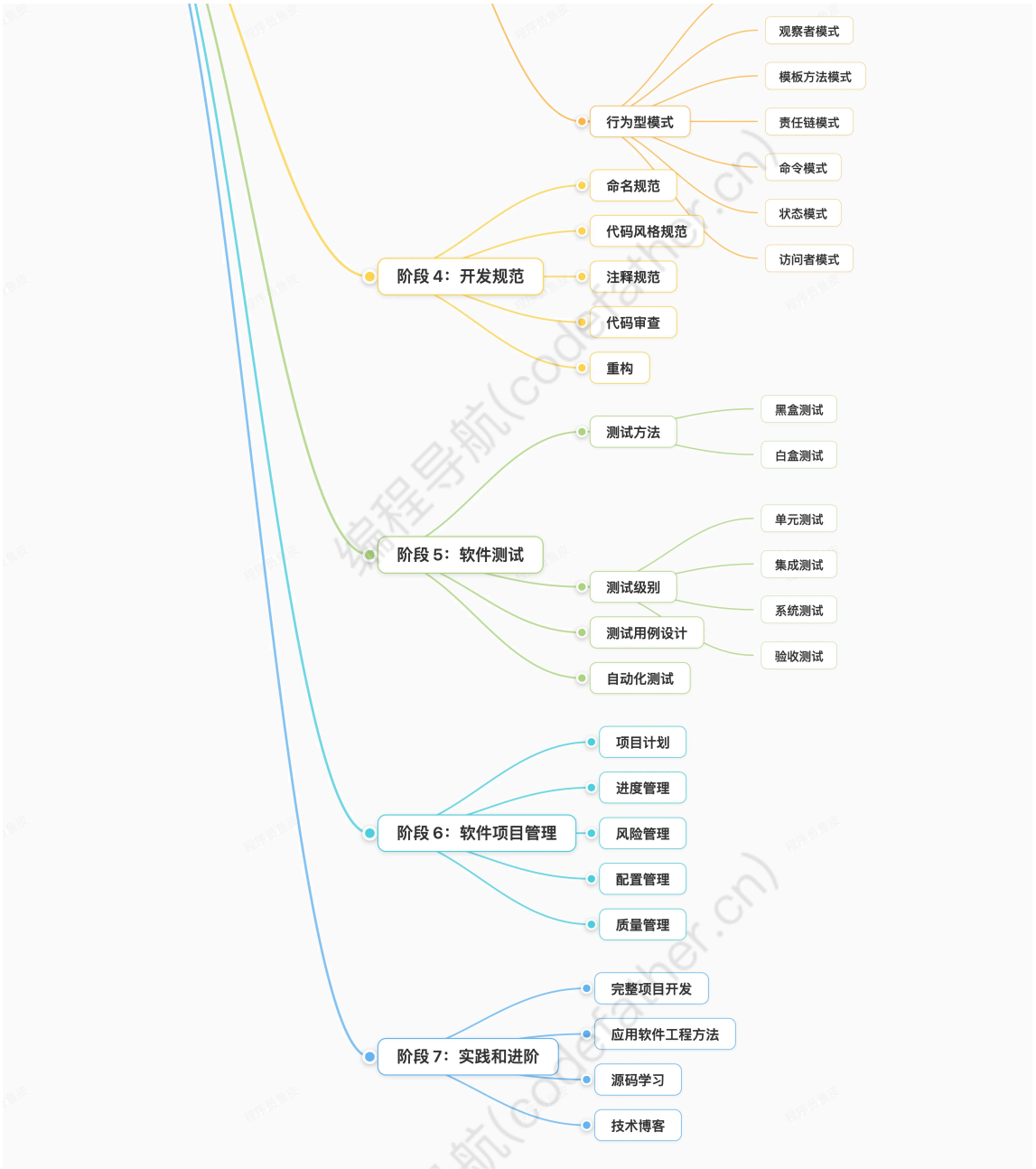
软件工程不是一个独立的岗位，而是软件开发、架构师、项目经理等岗位的核心能力。掌握软件工程后，可以从事：

1. 软件开发工程师：遵循软件工程规范开发高质量代码
2. 系统架构师：设计复杂系统的架构，应用软件工程原理
3. 项目经理 / 技术 Leader：管理团队和项目，确保开发流程规范
4. 质量保证工程师：负责软件质量管理和流程优化
5. DevOps 工程师：实现自动化开发、测试、部署流程

学习路线图

编程导航(codefather.cn)


软件工程学习路线
by 程序员鱼皮
阶段 1：软件工程基础
软件工程基本概念
软件的本质和特性
软件工程定义和目标
软件开发的挑战
软件危机
软件过程
软件生命周期
瀑布模型
软件过程模型
迭代模型
敏捷开发
DevOps
软件质量
软件质量定义
软件质量要素
如何保证软件质量
阶段 2：软件开发原则
SOLID 原则
单一职责原则 SRP
开闭原则 OCP
里氏替换原则 LSP
接口隔离原则 ISP
依赖倒置原则 DIP
其他开发原则
DRY
KISS
YAGNI
迪米特法则
组合优于继承
阶段 3：设计模式
创建型模式
单例模式
工厂方法模式
抽象工厂模式
建造者模式
原型模式
结构型模式
适配器模式
装饰器模式
代理模式
外观模式
桥接模式
组合模式
享元模式
策略模式



整体学习建议

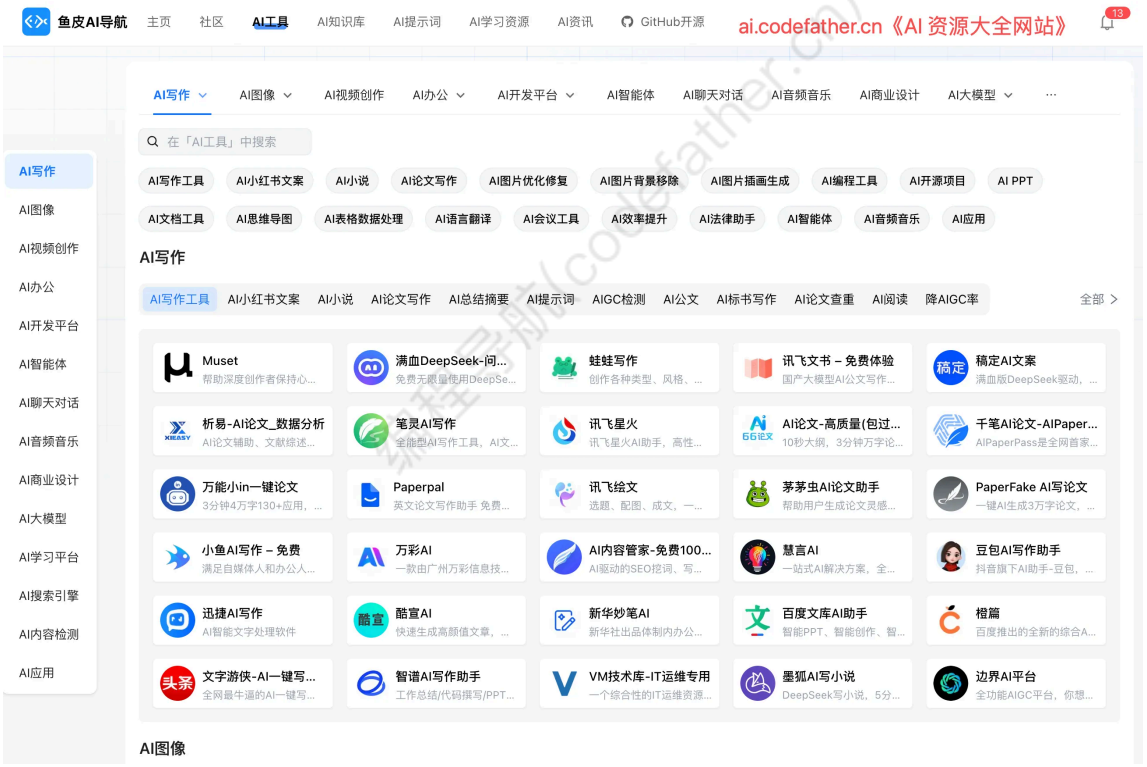
1）先做项目再学理论：软件工程的很多概念和方法只有在实践中才能理解。建议先独立完成 1-2 个项目，有了实际经验后再学习软件工程，会有 "醍醐灌顶" 的感觉。

2）重点学习实用内容：如果你是软件开发同学，优先学习开发原则、设计模式、开发规范、软件测试这些直接用得上的内容。其他理论知识可以简单了解。

3）不要死记硬背：软件工程的知识点很多，不要死记硬背。建议在实际开发中应用这些原则和方法，感受软件工程的有效性。

4）结合实际项目学习：建议在学习设计模式时，分析一些知名框架的源码（如 Spring、MyBatis），看看它们是如何应用设计模式的。关于设计模式的详细学习，可以查看 设计模式学习路线。

5）善用 AI 工具：学习软件工程时可以用 AI 工具（如 ChatGPT / 豆包）辅助理解概念、生成代码、分析设计方案。推荐使用 AI 资源大全 中的工具。



阶段 1：软件工程基础（3-5 天）

学习目标

理解软件工程的基本概念、特点和应用价值。

知识点

软件工程基本概念【必学】：

- 软件的本质和特性
- 软件工程的定义和目标
- 软件开发的挑战（复杂性、一致性、可变性、不可见性）
- 软件危机和软件工程的诞生

软件过程【必学】：

- 什么是软件过程
- 软件生命周期（需求、设计、编码、测试、维护）
- 软件过程模型
  - 瀑布模型
  - 迭代模型
  - 敏捷开发（Scrum、看板）【建议学】
  - DevOps【建议学】

软件质量【必学】：

- 软件质量的定义
- 软件质量要素（功能性、可靠性、易用性、效率、可维护性、可移植性）
- 如何保证软件质量

学习建议

1）软件工程的理论知识比较抽象，不要一开始就深入学习。建议快速过一遍基本概念，理解软件工程的核心价值即可。

2）敏捷开发是目前最主流的软件开发方法，要重点理解其核心思想：拥抱变化、快速迭代、持续交付。很多互联网公司都在使用敏捷开发。

3）这个阶段不需要深入学习每种过程模型的细节，重点理解不同模型的特点和适用场景即可。

学习资源

- 北京大学软件工程：深入浅出的软件工程课程
- 《软件工程：实践者的研究方法》：经典大黑书，内容全面

阶段 2：软件开发原则（5-7 天）

软件开发原则包括 SOLID、DRY、KISS 等设计原则，指导开发者编写高质量的代码。

学习目标

掌握软件开发的核心原则，能够在编码时应用这些原则。

知识点

编程导航(codefather.cn)

SOLID 原则【必学】：

- 单一职责原则（SRP）：一个类只负责一个职责
- 开闭原则（OCP）：对扩展开放，对修改关闭
- 里氏替换原则（LSP）：子类可以替换父类
- 接口隔离原则（ISP）：接口要小而精
- 依赖倒置原则（DIP）：依赖抽象而不是具体实现

其他开发原则【建议学】：

- DRY（Don't Repeat Yourself）：不要重复自己
- KISS（Keep It Simple, Stupid）：保持简单
- YAGNI（You Aren't Gonna Need It）：不要过度设计
- 迪米特法则（最少知道原则）
- 组合优于继承

学习建议

1）SOLID 原则是面向对象设计的核心原则，也是设计模式的理论基础。要深入理解每个原则的含义和应用场景。

2）这些原则不要死记硬背，要在实际编码中体会。比如当你发现一个类越写越长、职责越来越多时，就应该想到单一职责原则，考虑拆分。

3）开闭原则是最重要的原则，很多设计模式（如工厂模式、策略模式）都是为了实现开闭原则。要理解如何通过抽象和多态来实现 "对扩展开放，对修改关闭"。

4）DRY、KISS、YAGNI 这些原则虽然简单，但在实际开发中非常实用。很多同学容易过度设计，写出过于复杂的代码，要记住：简单就是美。

学习资源

- ⭐ 鱼皮的设计模式教程：包含 SOLID 原则详解
- 代码整洁之道：经典书籍，讲解编程原则

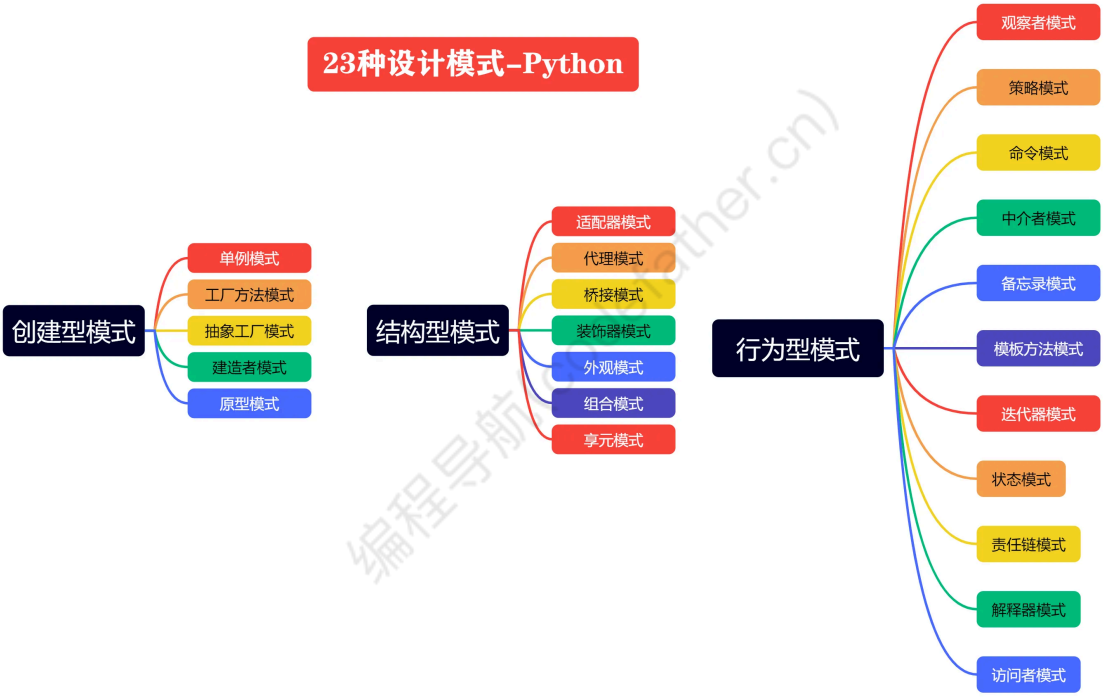
阶段 3：设计模式（15-30 天）

设计模式是软件设计中的经典解决方案，掌握常用设计模式能够提升代码的可维护性和可扩展性。

学习目标

掌握 23 种经典设计模式，能够在实际项目中灵活运用。

编程导航(codefather.cn)



编程导航(codefather.cn)

知识点

设计模式基础【必学】：

- 什么是设计模式
- 设计模式的分类（创建型、结构型、行为型）
- 设计模式和设计原则的关系

创建型模式【必学】：

- 单例模式：确保一个类只有一个实例
- 工厂方法模式：定义创建对象的接口
- 抽象工厂模式：创建相关对象家族
- 建造者模式：构建复杂对象
- 原型模式：通过复制创建对象【建议学】

结构型模式【必学】：

- 适配器模式：接口转换
- 装饰器模式：动态添加职责
- 代理模式：控制对象访问
- 外观模式：简化子系统接口
- 桥接模式：抽象和实现分离【建议学】
- 组合模式：树形结构处理【建议学】
- 享元模式：共享对象节省内存【可不学】

行为型模式【必学】：

- 策略模式：算法封装和切换
- 模板方法模式：定义算法骨架

- 观察者模式：对象间的依赖通知
- 责任链模式：请求传递处理
- 迭代器模式：遍历聚合对象
- 状态模式：对象状态变化【建议学】
- 命令模式：请求封装为对象【建议学】
- 访问者模式：操作对象结构【可不学】
- 中介者模式：对象间的中介【可不学】
- 备忘录模式：保存和恢复状态【可不学】
- 解释器模式：语言解释【可不学】

学习建议

1）设计模式是软件工程中最重要的内容，也是面试的高频考点。23 种设计模式不需要全部精通，建议重点学习常用的 10-12 种。

2）学习设计模式的顺序建议按使用频率：单例模式 → 工厂模式 → 策略模式 → 代理模式 → 观察者模式 → 模板方法模式 → 适配器模式 → 责任链模式。

3）每学一个设计模式，建议：先理解是什么和为什么 → 看 UML 类图理解结构 → 手写代码实现 → 分析框架源码中的应用。不要死记硬背代码。

4）鱼皮的设计模式教程非常详细，全面覆盖 23 种设计模式，包含 Java 示例代码、UML 类图、使用场景详解和源码解析，强烈推荐。

5）学完基础的设计模式后，建议阅读 Spring、MyBatis 等框架的源码，看看大厂是如何应用设计模式的。这会让你对设计模式有更深的理解。

6）**不要过度使用设计模式！** 设计模式不是银弹，过度使用只会增加系统复杂度。要根据实际需求选择合适的设计模式。

经典面试题

1. 什么是设计模式？设计模式分为哪几类？
2. 请说说你了解的常见设计模式及其应用场景
3. 单例模式有哪几种实现方式？各有什么优缺点？
4. 工厂模式和抽象工厂模式有什么区别？
5. 策略模式和状态模式有什么区别？
6. Spring 框架中使用了哪些设计模式？
7. 装饰器模式、适配器模式、代理模式、桥接模式有什么区别？

学习资源

- ⭐ 鱼皮的设计模式教程：23 种设计模式完整教程
- ⭐ 鱼皮的设计模式 GitHub：开源设计模式教程
- 图解设计模式：生动形象的设计模式讲解
- Head First 设计模式：经典设计模式书籍
- 设计模式面试题 - 面试鸭：全面的设计模式面试题

阶段 4：开发规范（3-7 天）

开发规范包括代码规范、版本管理、文档编写等内容，是团队协作的基础。

学习目标

掌握代码规范、文档规范、Git 规范等开发规范，养成良好的编程习惯。

知识点

代码规范【必学】：

- 命名规范（变量、函数、类、包等）
- 代码格式规范（缩进、空格、换行等）
- 注释规范（单行注释、多行注释、文档注释）
- 编码原则（SOLID、DRY、KISS 等）
- 异常处理规范
- 并发编程规范【建议学】

文档规范【建议学】：

- 需求文档（PRD）
- 设计文档（架构设计、详细设计）
- 接口文档（API 文档）
- README 文档
- 变更日志（CHANGELOG）

Git 规范【必学】：

- 分支管理规范（Git Flow、GitHub Flow）
- Commit 提交规范（Conventional Commits）
- 代码审查规范（Code Review）
- Pull Request 规范

代码检查工具【必学】：

- Java：CheckStyle、SonarQube、SpotBugs
- Python：Pylint、Flake8、Black
- JavaScript/TypeScript：ESLint、Prettier
- Go：golint、gofmt

学习建议

1）开发规范是整个软件工程中最重要、最应该优先学习的内容！因为：**开发不规范，同事两行泪！**

遵守规范有利于提高项目的开发效率、降低维护成本。

2）每个团队的开发规范可能不完全相同，千万不要死记硬背！建议先过一遍大厂的代码规范手册（如阿里巴巴 Java 开发手册），做项目时想到了某个规范，就去查阅确认。

3）善用代码检查工具。现代 IDE（如 IDEA、VSCode）都集成了代码检查功能，会自动提示不规范的代码。建议多观察编辑器给出的警告，尝试消除它们。

4）项目做得多了、慢慢养成习惯后，你会发现遵循规范是一件 "水到渠成" 的事情，不用刻意去记。

学习资源

- ⭐ 阿里巴巴 Java 开发手册：搜索 "Java 开发手册"
- ⭐ 华山版《Java 开发手册》讲解：官方讲解视频
- Google Java Style Guide：Google Java 代码规范
- Airbnb JavaScript 代码规范：前端代码规范
- Conventional Commits：Git 提交规范

阶段 5：软件测试（3-7 天）

编程导航(codefather.cn)

软件测试是保证软件质量的重要手段，包括单元测试、集成测试、系统测试等。

学习目标

掌握软件测试的基本方法，能够编写单元测试和集成测试。

知识点

测试基础【必学】：

- 软件测试的目的和意义
- 测试的分类（黑盒测试、白盒测试）
- 测试的层次（单元测试、集成测试、系统测试、验收测试）

单元测试【必学】：

- 什么是单元测试
- 单元测试框架（JUnit、PyTest、Jest 等）
- 测试覆盖率（Jacoco）
- Mock 和 Stub【建议学】

集成测试【建议学】：

- 什么是集成测试
- 集成测试的策略
- 接口测试工具（Postman、JMeter）

自动化测试【建议学】：

- 测试自动化的价值
- 持续集成（CI）和持续测试
- 自动化测试框架（Selenium、Cypress）

性能测试【可不学】：

- 压力测试、负载测试
- 性能测试工具（JMeter、Gatling）

学习建议

1）单元测试是开发者的基本功，要重点掌握。很多同学觉得写单元测试浪费时间，但实际上单元测试能帮你发现代码中的问题，提高代码质量。

2）测试覆盖率不是越高越好，要关注核心业务逻辑的测试。100% 的测试覆盖率往往意味着很多无意义的测试。

3）Mock 和 Stub 是单元测试中的重要技术，用于隔离外部依赖（如数据库、网络请求）。要理解它们的概念和使用场景。

4）集成测试和系统测试一般由专业的测试工程师负责，开发者了解基本概念即可。

学习资源

- 《软件测试》（Ron Patton 著）：经典测试书籍

阶段 6：软件项目管理（选学，3-5 天）

软件项目管理包括需求管理、进度管理、风险管理等内容，是项目成功的关键因素。

编程导航(codefather.cn)

学习目标

了解软件项目管理的基本概念和方法，理解敏捷开发流程。

知识点

项目管理基础【建议学】：

- 项目管理的定义和目标
- 项目管理的要素（范围、时间、成本、质量）
- 项目管理的过程（启动、规划、执行、监控、收尾）

敏捷项目管理【建议学】：

- Scrum 框架（Sprint、Daily Standup、Sprint Review）
- 看板方法（Kanban）
- 用户故事（User Story）
- 迭代计划和估算

项目管理工具【可不学】：

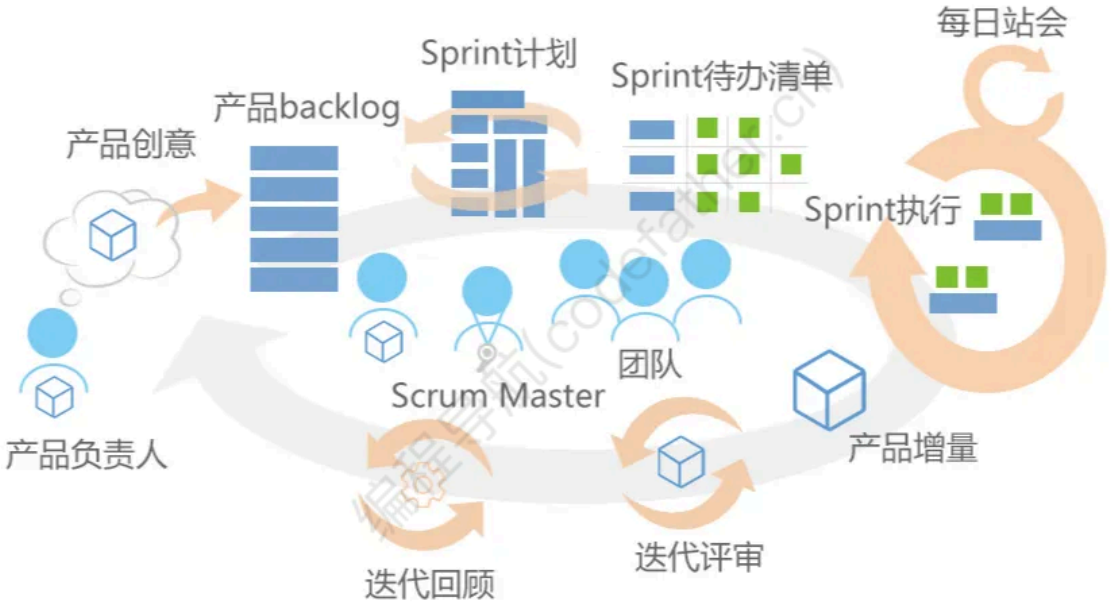
- Jira：最流行的项目管理工具
- 禅道：国产项目管理工具
- Trello：轻量级看板工具
- Teambition：阿里的协作工具

学习建议

1）项目管理对于普通开发者来说不是必学的，但对于 Team Leader、项目经理来说是必备技能。如果你未来想往管理方向发展，建议学习。

2）Scrum 是最流行的敏捷框架，要理解其核心概念：Sprint（冲刺）、Product Backlog（产品待办列表）、Sprint Planning（迭代计划会）、Daily Standup（每日站会）等。

编程导航(codefather.cn)
编程导航(codefather.cn)



3）项目管理工具的使用很简单，不需要专门学习。入职后跟着团队使用即可。

学习资源

- 《敏捷软件开发》（Robert C. Martin 著）：敏捷开发经典书籍
- 《人月神话》（Frederick P. Brooks Jr. 著）：软件项目管理经典

阶段 7：实践和进阶（持续学习）

学习目标

将软件工程知识应用到实际项目中，持续提升软件工程能力。

知识点

需求工程【可不学】：

- 需求获取和分析
- 需求规格说明
- 需求验证

UML 建模【可不学】：

- UML 图的类型（类图、时序图、用例图等）
- UML 建模工具（PlantUML、draw.io）

软件架构【建议学】：

- 分层架构、MVC 架构
- 微服务架构
- 事件驱动架构

编程导航(codefather.cn)

- 领域驱动设计（DDD）【建议学】

DevOps 实践【建议学】：

- 持续集成（CI）
- 持续交付（CD）
- 自动化部署
- 基础设施即代码（IaC）【可不学】

学习建议

1）软件架构是高级内容，建议有一定项目经验后再学习。可以参考 后端系统设计学习路线。

2）领域驱动设计（DDD）是一种复杂系统的设计方法论，强调业务领域建模。鱼皮的智能协同云图库项目就采用了 DDD 架构设计，可以通过 云图库项目 实战学习 DDD 的应用。

3）DevOps 是现代软件开发的重要实践，建议了解 CI/CD 的基本概念。可以参考 DevOps 学习路线 和 CI/CD 学习路线。

4）UML 建模在实际工作中使用频率不高，简单了解即可。更多时候我们用流程图、架构图等更直观的方式来表达设计。

学习资源

- 后端系统设计学习路线：深入学习系统设计
- ⭐ 编程导航 - 智能协同云图库项目：DDD 领域驱动设计实战
- DevOps 学习路线：了解 DevOps 实践
- CI/CD 学习路线：学习持续集成和持续交付

持续学习资源

经典书籍

- ⭐ 《软件工程：实践者的研究方法》：经典大黑书，内容全面（但比较厚重）
- ⭐ 《代码整洁之道》：教你写出整洁易读的代码
- 《软件开发的 201 个原则》：工具书，实用性强
- 《重构：改善既有代码的设计》：经典重构书籍
- 《设计模式：可复用面向对象软件的基础》：设计模式圣经（GoF）

知识总结

- ⭐ 编程导航：学习路线、项目教程、面试题、编程资源一站式平台
- 设计模式学习路线：专门的设计模式学习路线
- 后端系统设计学习路线：系统设计方法论

技术博客

- Martin Fowler Blog：软件工程大师博客
- Google Engineering Practices：谷歌工程实践
- Spotify Engineering：Spotify 工程文化
- Netflix TechBlog：Netflix 工程实践

写在最后

软件工程是一门实践性很强的学科，理论学习只是第一步，更重要的是在实际项目中应用。

建议大家在学习软件工程时，不要追求面面俱到，重点掌握软件开发原则、设计模式、开发规范、软件测试这些最实用的内容。当你成为团队的负责人后，再去深入学习项目管理、需求工程等内容。

希望这份学习路线能够帮助大家系统地掌握软件工程知识，写出更高质量的代码，成为更优秀的软件工程师。

加油小伙伴们 💪！



编程导航(codefather.cn)

程序员必备资源

1）程序员学习交流圈：极客教程、实战项目、求职宝典

2）程序员面试八股文：实习/校招/社招高频考点、企业真题解析

3）程序员写简历神器：专业模板、丰富例句、直通面试

4）AI 知识资源大全：前沿技术、最新 AI 资讯、提示词大全

5）1 对 1 模拟面试：实习/校招/社招面试拿 Offer 必备

编程导航(codefather.cn)